Gerência de configuração

Introdução ao Kubernetes

Prof. Diego Cardoso Borda Castro

Introdução

- Container
  - Executar processos de forma isolada
  - Sem compartilhar recursos
  - Ambiente escalável?
    - Vertical
    - Horizontal
      - Alta disponibilidade
      - Réplicas das aplicações (Cópia)
      - Balanceador de carga
      - Self healing
      - Atualização

Orquestradores

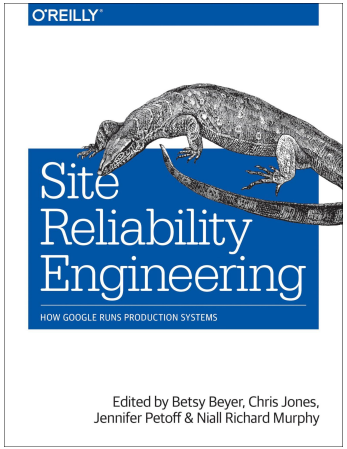
Introdução

- Orquestradores
  - Docker Swarm
  - Apache Mesos
    - Marathon
  - Hashikorp Nomad
  - Provedores de cloud
  - Kubernetes

Capability	Docker Swarm	Kubernetes
Setup Summary	Yes	Yes
Role-driven Access	Yes	Yes
Automatic Scaling	No	Yes
Load Balancing	No	Yes
Continuous Updates	Yes	Yes
Supervision of Multiple Clusters	No	Yes (with third-party VCS tools)
Live Log Monitoring	No	Yes (with third-party VCS tools)
Advanced Troubleshooting Capabilities	No	Yes (with third-party VCS tools)

Introdução

- Criado em 2014 pela google
  - Antes vieram o Borg e Omega
- Foi aberto e doado para o linux
- O nome Kubernetes vem do grego
  - “timoneiro” ou “piloto”
  - “K” + 8 letras omitidas + “s”
- Fazia parte de um conjunto de 9 projetos de containers da google
  - Leme com 7 pontas



Introdução

- Funcionalidades
  - Escalar verticalmente e horizontalmente
  - Service discovery
  - Balanceamento de carga
  - Containers
    - Pods
  - Gerenciamento de configuração baseados em arquivos
  - Gerenciamento de atualização
  - Self healing

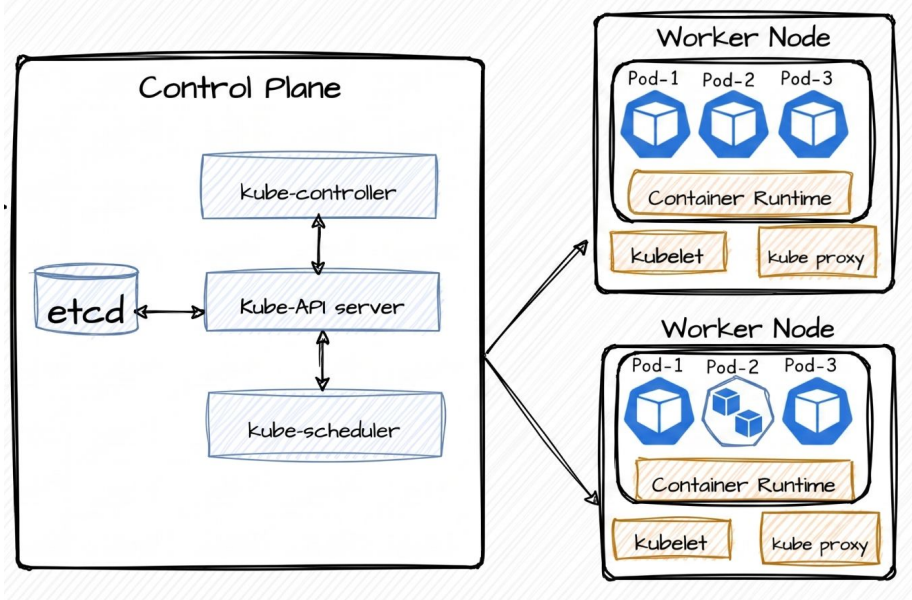
Introdução

- Linux Foundation
- Cloud Native Computing Foundation
  - Ferramentas agnósticas
    - https://landscape.cncf.io/
  - Certificações



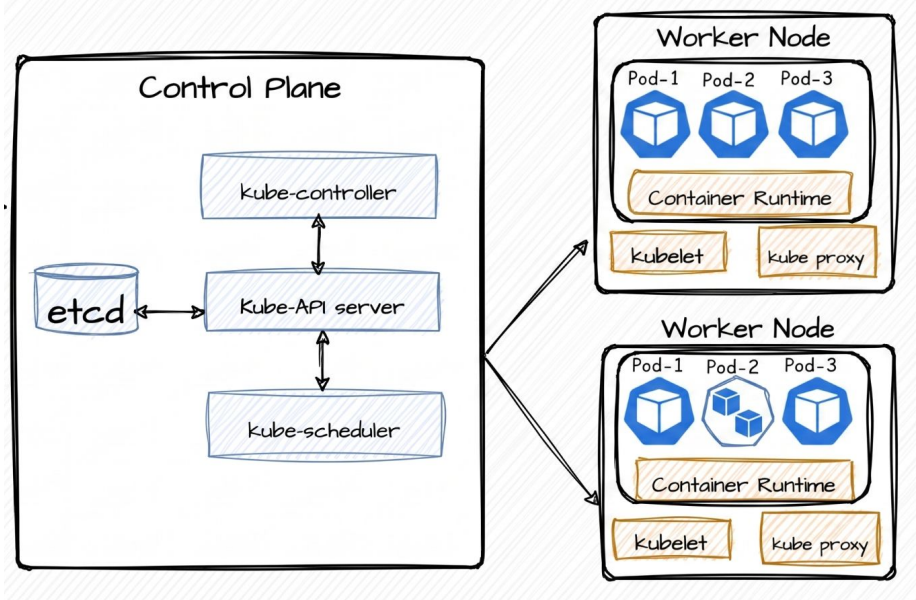
Arquitetura e conceitos

- Control plane
  - API Server
    - Porta de entrada do cluster
  - Scheduler
    - Decide em qual node cada container (Pod) será executado
  - Etcd
    - Armazena o estado do cluster



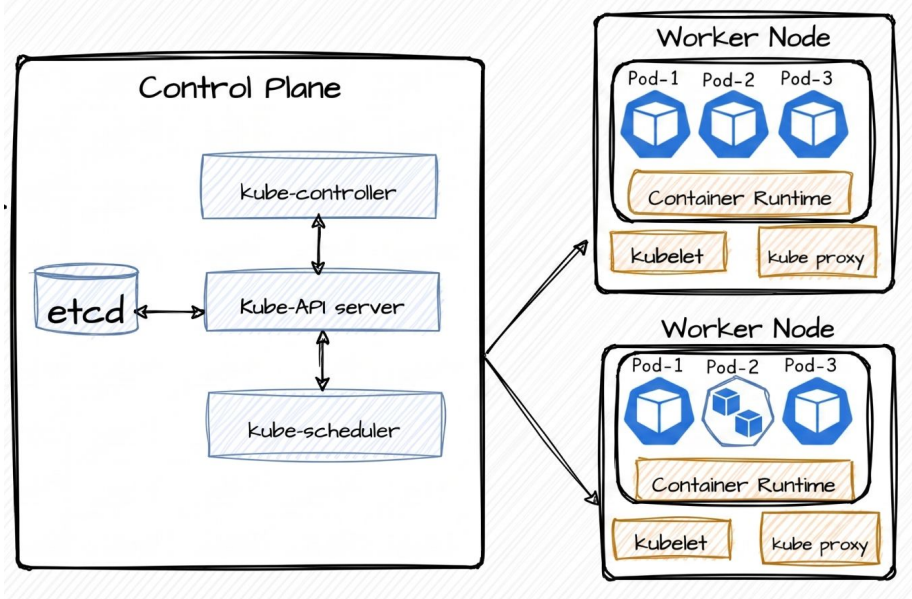
Arquitetura e conceitos

- Controller Manager
  - Garante que o estado desejado seja mantido
    - Ex: recriar Pods que caíram
  - Controladores
    - Node Controller
    - Deployment
    - ReplicaSet
    - Job
    - Namespace
    - …



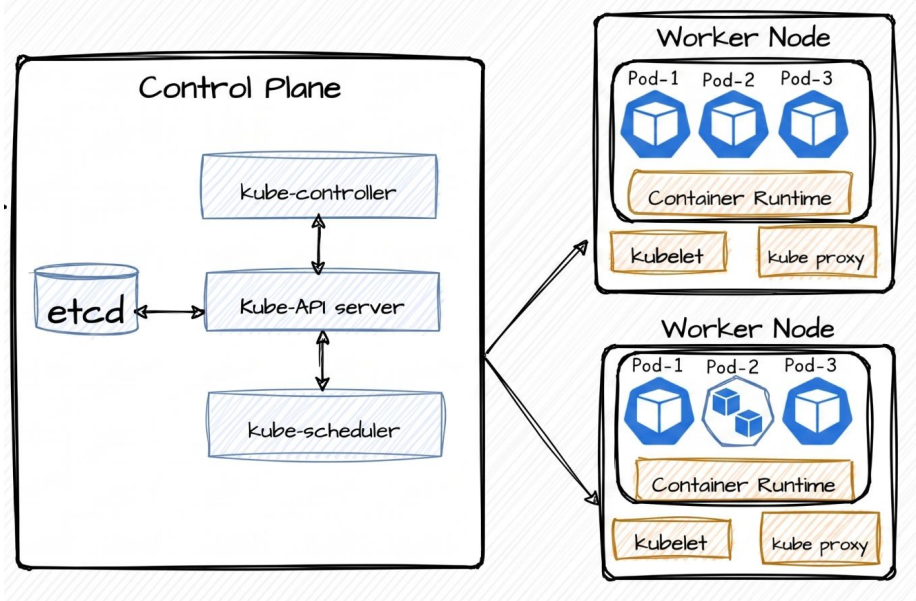
Arquitetura e conceitos

- Worker node
  - Kubelet
    - Agente que executa nos nodes
    - Garante que os Pods estão rodando corretamente
  - Kube-proxy
    - Gerencia rede e comunicação
    - Faz balanceamento de tráfego



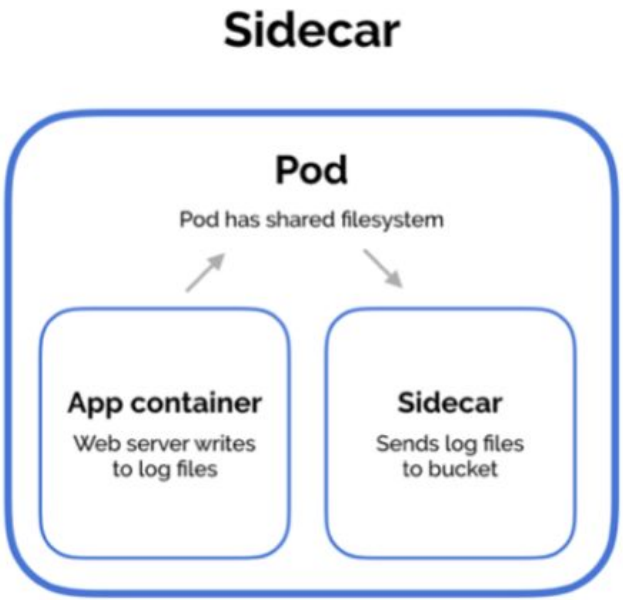
Arquitetura e conceitos

- Container Runtime
  - Responsável por rodar containers
    - Ex: containerd
  - Cada Pod pode ter 1 ou mais containers
  - O runtime é quem cria e roda esses containers



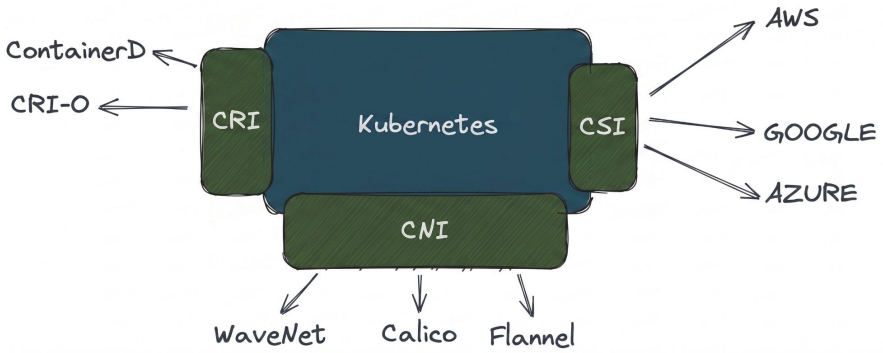
Arquitetura e conceitos

- POD
  - Onde nós iremos executar nossos containers
  - O POD é o menor elemento da estrutura kubernetes
    - Ele que vai escalar
      - Um POD pode conter mais de um container
      - Mas existem padrões como sidecar que favorecem a utilização de apenas um container por POD



Arquitetura e conceitos

- Interfaces
  - Container Runtime Interface
    - Antes era o docker
  - Container Network Interface
  - Container Storage Interface
    - Longhorn
    - Rook
    - OpenEBS

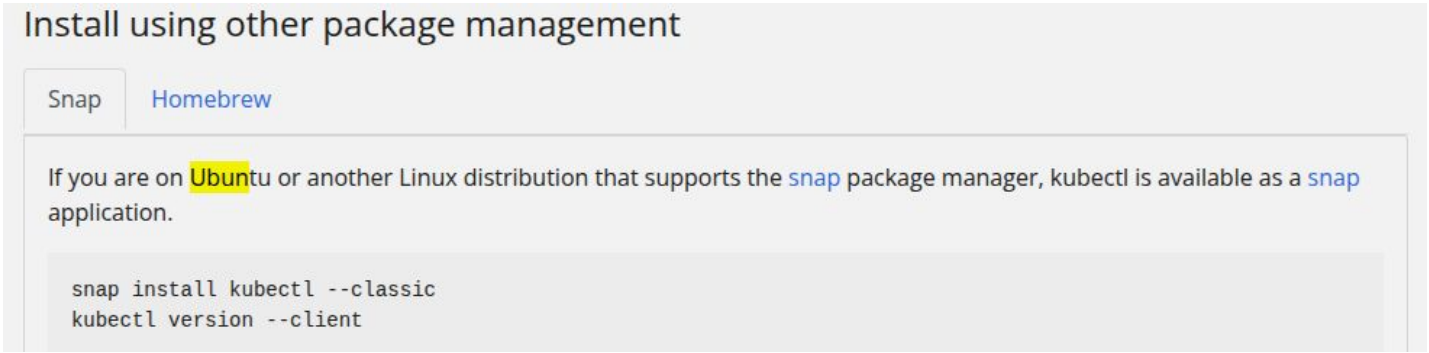


Criando nosso primeiro cluster

- Formas de criar cluster kubernetes
  - On-premise
    - Da mais trabalho
    - Total controle sobre o ambiente
    - KubeAdm, RKE, KubeSpray, K3S, MicroK8S, …
  - Cloud provider
    - PaaS
    - Vai acessar via cliente (KubeClt)
    - AKS (Azure), EKS (AWS), GKE (Google), OKE (Oracle)
  - Localmente (Estudo e PoC)
    - Minikube, K3D, Kind, …

Criando nosso primeiro cluster

- Ferramentas para rodar nosso ambiente local para aprendizado
  - https://kubernetes.io/docs/tasks/tools/
  - Instalar o Kubectl
    - Ferramenta de linha de comando para interagir com o cluster
    - Vai se comunicar com a API



Criando nosso primeiro cluster

- Ferramentas para rodar nosso ambiente local para aprendizado
  - https://k3d.io/stable/
  - k3d cluster create
    - --servers 1
    - -- agents 2
  - kubectl cluster-info
  - kubectl get nodes
  - k3d cluster delete nome



Criando nosso primeiro cluster

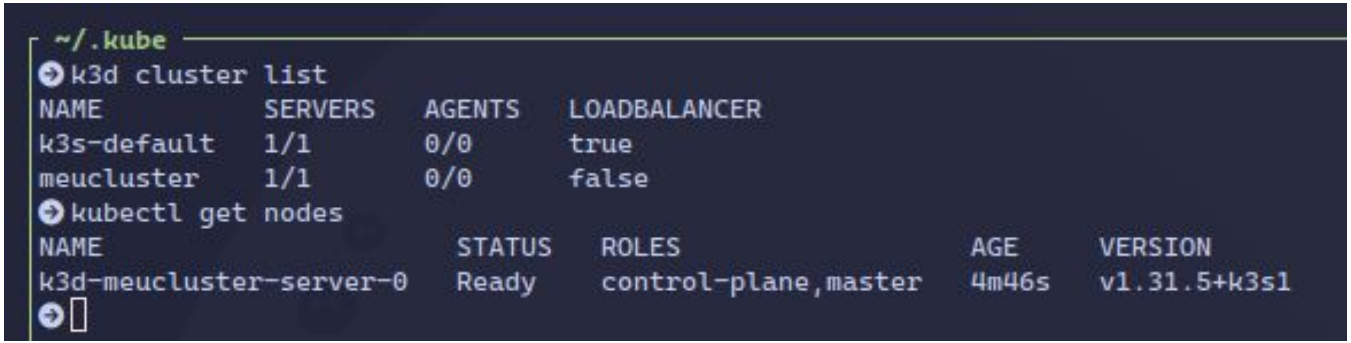
```
~/.kube
kubectl get nodes
NAME                        STATUS   ROLES                  AGE     VERSION
k3d-k3s-default-server-0    Ready    control-plane,master   9m35s   v1.31.5+k3s1
docker container ls
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          P
ORTS                              NAMES
d2b5101c1e88   ghcr.io/k3d-io/k3d-proxy:5.8.3       "/bin/sh -c nginx-pr…"   12 minutes ago   Up 12 minutes   8
0/tcp, 0.0.0.0:45471->6443/tcp    k3d-k3s-default-serverlb
6dbb4665287a   rancher/k3s:v1.31.5-k3s1             "/bin/k3d-entrypoint…"   12 minutes ago   Up 12 minutes
                                  k3d-k3s-default-server-0
k3d cluster list
NAME          SERVERS   AGENTS   LOADBALANCER
k3s-default   1/1       0/0      true
```

Criando nosso primeiro cluster

```
k3d cluster create meucluster --no-lb
INFO[0000] Prep: Network
INFO[0000] Created network 'k3d-meucluster'
INFO[0000] Created image volume k3d-meucluster-images
INFO[0000] Starting new tools node...
INFO[0000] Starting node 'k3d-meucluster-tools'
INFO[0001] Creating node 'k3d-meucluster-server-0'
INFO[0001] Using the k3d-tools node to gather environment information
INFO[0001] HostIP: using network gateway 172.26.0.1 address
INFO[0001] Starting cluster 'meucluster'
INFO[0001] Starting servers...
INFO[0001] Starting node 'k3d-meucluster-server-0'
INFO[0006] All agents already running.
INFO[0006] All helpers already running.
INFO[0006] Injecting records for hostAliases (incl. host.k3d.internal) and for 1 network members into CoreD
NS configmap...
INFO[0008] Cluster 'meucluster' created successfully!
INFO[0008] You can now use it like this:
kubectl cluster-info
k3d cluster list
NAME          SERVERS   AGENTS   LOADBALANCER
k3s-default   1/1       0/0      true
meucluster    1/1       0/0      false
```

Criando nosso primeiro cluster

- Pq só mostrou 1 node?



Criando nosso primeiro cluster

- O K3d só vê um cluster po vez

```
~/.kube
kubectl config get-contexts
CURRENT   NAME              CLUSTER           AUTHINFO                NAMESPACE
          k3d-k3s-default   k3d-k3s-default   admin@k3d-k3s-default
*         k3d-meucluster    k3d-meucluster    admin@k3d-meucluster
kubectl config use-context k3d-meucluster
Switched to context "k3d-meucluster".
kubectl get nodes
NAME                     STATUS   ROLES                  AGE     VERSION
k3d-meucluster-server-0   Ready    control-plane,master   6m43s   v1.31.5+k3s1
kubectl config use-context k3d-k3s-default
Switched to context "k3d-k3s-default".
kubectl get nodes
NAME                     STATUS   ROLES                  AGE   VERSION
k3d-k3s-default-server-0   Ready    control-plane,master   25m   v1.31.5+k3s1
```

Criando nosso primeiro cluster

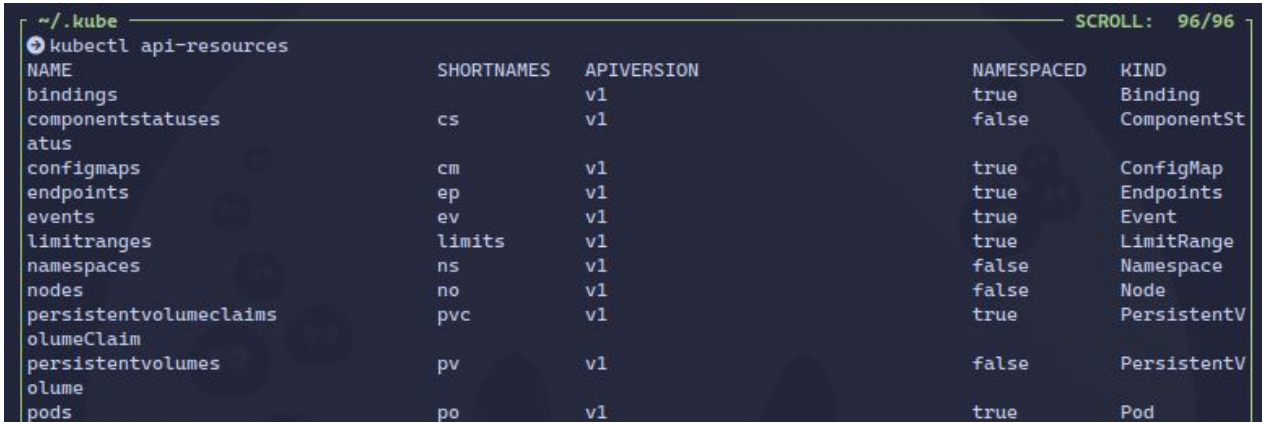
```
k3d cluster create meucluster --servers 1 --agents 3 -p "8080:30000@loadbalancer"
INFO[0000] portmapping '8080:30000' targets the loadbalancer: defaulting to [servers:*:proxy agents:*:proxy]
INFO[0000] Prep: Network
INFO[0000] Re-using existing network 'k3d-meucluster' (b13d330450528e54473b37f5771790947071b3f845abac4d8ca44b
```

```
kubectl get nodes
NAME                       STATUS   ROLES
k3d-meucluster-agent-0     Ready    <none>
k3d-meucluster-agent-1     Ready    <none>
k3d-meucluster-agent-2     Ready    <none>
k3d-meucluster-server-0    Ready    control-pla
```

```
k3d cluster list
NAME          SERVERS   AGENTS   LOADBALANCER
meucluster    1/1       3/3      true
```

Criando nosso primeiro cluster

- Arquivo manifesto
  - Arquivo de configuração do que será executado no seu cluster
  - Arquivo YAML
  - Criação declarativa



Criando nosso primeiro cluster

- Arquivo manifesto
  - apiVersion
    - v1 → recursos básicos (Pod, Service)
    - apps/v1 → Deployments, ReplicaSets
    - batch/v1 → Jobs, CronJobs
  - Kind
    - Tipo do recurso que será criado
    - Pod, Deployment, Service, ConfigMap, Secret, Ingress, …
  - Metadata
    - Contém informações descritivas do recurso
    - name, namespace, label, annotation

Criando nosso primeiro cluster

- Arquivo manifesto
  - spec
    - define o comportamento do recurso
    - ports:
      - - containerPort:
    - env:
      - - name: VAR
      - value: "valor"

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: meu-app
  template:
    metadata:
      labels:
        app: meu-app
    spec:
      containers:
        - name: app
          image: nginx
```

Criando nosso primeiro cluster

- spec
  - resources:
    - limits:
      - cpu: "500m"
      - memory: "256Mi"
    - requests:
      - cpu: "250m"
      - memory: "128Mi"
- volumeMounts:
  - - mountPath: /data
  - name: volume1

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: meu-app
  template:
    metadata:
      labels:
        app: meu-app
    spec:
      containers:
        - name: app
          image: nginx
```

Criando nosso primeiro cluster

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: node-app
  labels:
    app: node-app
spec:
  imagePullSecrets:
    - name: dockerhub-secret
  containers:
    - name: node
      image: diegocbcastroo/node-app-exemplo:1.0
      ports:
        - containerPort: 3000
```

```
kubectl create secret docker-registry dockerhub-secret \
  --docker-username=SEU_USUARIO \
  --docker-password=SUA_SENHA \
  --docker-email=diego.castro@cefet-rj.br
```

```
kubectl delete pods --all
pod "node-app-69b4bb8588-gkh55" deleted from default namespace
pod "node-app-69b4bb8588-li2dq" deleted from default namespace
kubectl apply -f pod.yaml
deployment.apps/node-app configured
```

```
kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
node-app-58b7c7bb9d-7zz4r   1/1     Running   0          2m19s
node-app-58b7c7bb9d-k4p6f   1/1     Running   0          2m18s
kubectl describe pod node-app-58b7c7bb9d-7zz4r
Name:             node-app-58b7c7bb9d-7zz4r
Namespace:        default
Priority:         0
Service Account:  default
```

Criando nosso primeiro cluster

Naked pod

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: node-app
  labels:
    app: node-app
spec:
  imagePullSecrets:
    - name: dockerhub-secret
  containers:
    - name: node
      image: diegocbcastroo/node-app-exemplo:1.0
      ports:
        - containerPort: 3000
```

```
kubectl get pods
NAME       READY   STATUS    RESTARTS   AGE
node-app   1/1     Running   0          5m10s
kubectl port-forward pod/node-app 3001:3000
Forwarding from 127.0.0.1:3001 -> 3000
Forwarding from [::1]:3001 -> 3000
Handling connection for 3001
Handling connection for 3001
Handling connection for 3001
```



Bem-vindo a API da sua primeira pipeline!

Estrutura hierárquica

- Por que NÃO usar Pod direto?
  - Sem alta disponibilidade
  - Sem escala
  - Sem auto-recuperação
  - Sem gerenciamento de versão
- Controllers
  - ReplicaSet
  - Deployment

```
kubectl delete pods --all
pod "node-app" deleted from default namespace
kubectl apply -f replicaSet.yaml
replicaset.apps/node-rs created
kubectl get replicasets
NAME      DESIRED   CURRENT   READY   AGE
node-rs   2         2         2       13s
kubectl get pods
NAME            READY   STATUS    RESTARTS   AGE
node-rs-q8mpv   1/1     Running   0          26s
node-rs-trxd9   1/1     Running   0          26s
```

Estrutura hierárquica

- Label
  - Par chave/valor que você coloca em qualquer recurso
- Selector
  - Mecanismo de busca baseado em labels

pod
labels:
app: node-app

Service
selector:
app: node-app

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: node-rs
spec:
  replicas: 2
  selector:
    matchLabels:
      app: node-app
  template:
    metadata:
      labels:
        app: node-app
    spec:
      containers:
        - name: node
          image: diegocbcastroo/node-app-exemplo:1.0
```

Estrutura hierárquica

- ReplicaSet
  - Te dá escabilidade
  - Mas não te o pod novo com a nova imagem

```
15        containers:
16          - name: node
17            image: diegocbcastroo/node-app-deno:1.0

PROBLEMS 1   OUTPUT   DEBUG CONSOLE   PORTS   TERMINAL   SPE

kubectl apply -f replicaSet.yaml
replicaset.apps/node-rs configured
kubectl get pods
NAME            READY   STATUS    RESTARTS      AGE
node-rs-9vhtl   1/1     Running   2 (11h ago)   20h
node-rs-th526   1/1     Running   2 (11h ago)   20h
```

```
IPs:
  IP:           10.42.0.6
Controlled By:  ReplicaSet/node-rs
Containers:
  node:
    Container ID:   containerd://5ee48dde6cb19b6710fe977d
    Image:          diegocbcastroo/node-app-exemplo:1.0
    Image ID:       docker.io/diegocbcastroo/node-app-exe
875020ac6040b
    Port:           <none>
    Host Port:      <none>
    State:          Running
```

Estrutura hierárquica

- Deployment
  - Camada acima do ReplicaSet
    - Atualizações
    - Rollback
    - Escala
    - Versionamento

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: node-app
  template:
    metadata:
      labels:
        app: node-app
    spec:
      containers:
        - name: node
          image: diegocbcastroo/node-app-exemplo:1.0
```

Estrutura hierárquica

- Service
  - Pods são efêmeros
    - IP muda toda hora
  - Service prove um acesso estável
  - Balanceador de carga

```yaml
apiVersion: v1
kind: Service
metadata:
  name: node-service
spec:
  type: NodePort
  selector:
    app: node-app
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30007
```

localhost:30007 → Service → Pod:3000

Estrutura hierárquica

- Tipos de services
  - ClusterIP
  - NodePort
  - Load Balance
  - External name

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: node-app
  template:
    metadata:
      labels:
        app: node-app
    spec:
      containers:
        - name: node
          image: diegocbcastroo/node-app-exemplo:1.0
---
apiVersion: v1
kind: Service
metadata:
  name: node-service
spec:
  type: ClusterIP
  selector:
    app: node-app
  ports:
    - protocol: TCP
      port: 3001
      targetPort: 3000
```

Estrutura hierárquica

```
kubectl run test --image=busybox:1.28 -it --rm -- sh
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
/ #
```

```
kubectl get pods -o wide
NAME           READY   STATUS    RESTARTS   AGE   IP           NODE                      READINESS GATES
node-rs-8n6ww  1/1     Running   0          41m   10.42.1.8    k3d-meucluster-agent-1    <none>
node-rs-dszs4  1/1     Running   0          38m   10.42.0.9    k3d-meucluster-agent-0    <none>
pod-teste      1/1     Running   0          17m   10.42.3.9    k3d-meucluster-server-0   <none>
test           1/1     Running   0          71s   10.42.0.10   k3d-meucluster-agent-0    <none>
```

- Todos no mesmo cluster
  - Sendo comunicados pelo service

```
PROBLEMS 2  OUTPUT  DEBUG CONSOLE  PORTS  TERMI

/ # wget -qO- http://node-service:3001
Bem-vindo a API da sua primeira pipeline!/ #
```

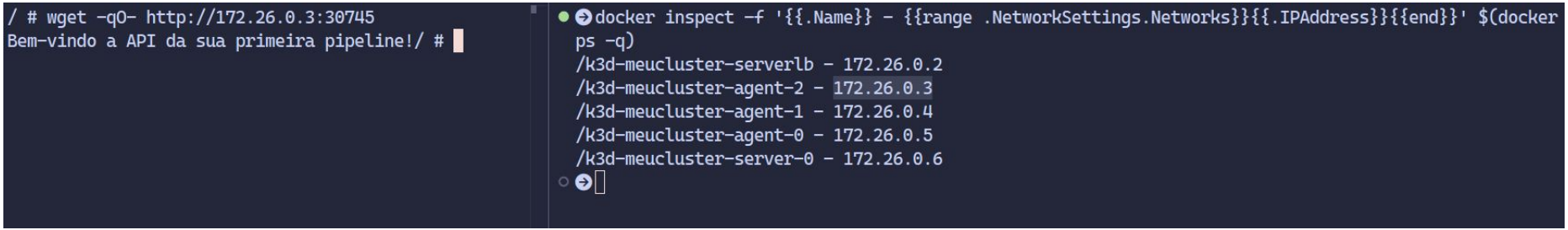
Estrutura hierárquica

- Nodeport terá que usar o IP de alguma máquina do cluster
  - Vai selecionar uma porta da 30000 até a 32767

```
kubectl apply -f
clusterIP.yaml  nodePorts.yaml
kubectl apply -f nodePorts.yaml
deployment.apps/node-deployment unchanged
service/node-service configured
kubectl get svc
NAME           TYPE        CLUSTER-IP      EXTERN
AL-IP   PORT(S)          AGE
kubernetes     ClusterIP   10.43.0.1       <none>
        443/TCP          23h
node-service   NodePort    10.43.242.221   <none>
        3001:30745/TCP   117m
```

Estrutura hierárquica

- Como estamos usando o K3D
  - Precisamos pegar o IP do container
  - nodePort



Estrutura hierárquica

- Load Balance
  - Cria um LB na frente do service para balancer o envio de dados para os pods
  - IP público de cloud
    - Amazon EKS
  - MetalLB

```yaml
apiVersion: v1
kind: Service
metadata:
  name: node-service-lb
spec:
  type: LoadBalancer
  selector:
    app: node-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3000
```

Estrutura hierárquica

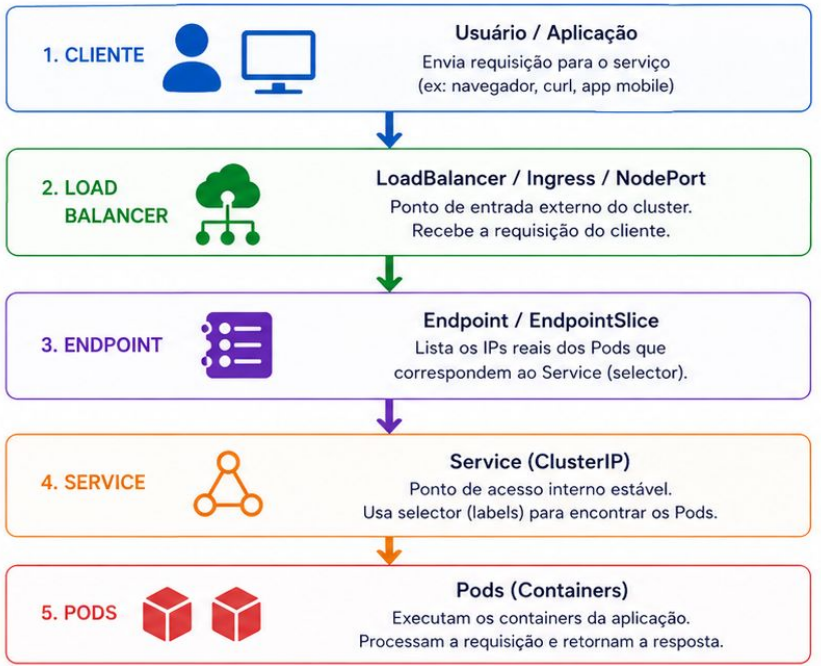
- ExternalName
  - Um Service que não aponta para Pod
  - Ele redireciona para um DNS externo
  - Quando usar
    - Acessar APIs externas
    - Integrar sistemas
    - Evitar hardcode de URLs



```yaml
apiVersion: v1
kind: Service
metadata:
  name: google-service
spec:
  type: ExternalName
  externalName: google.com
```

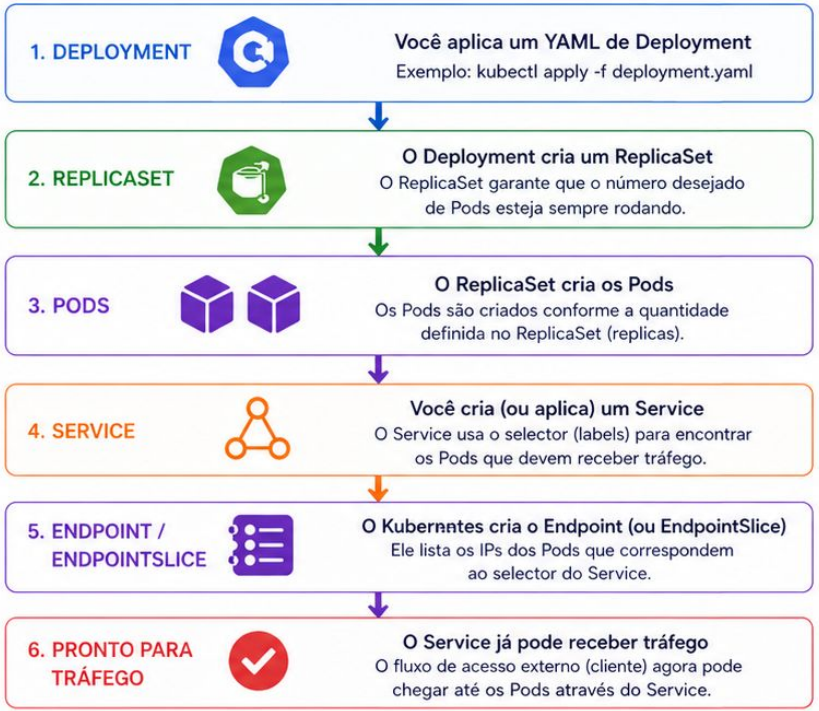
Estrutura hierárquica

- Endpoint
  - Lista de IPs reais dos Pods
  - kubectl get endpoints
    - node-service: 10.42.0.5:3000, 10.42.0.6:3000

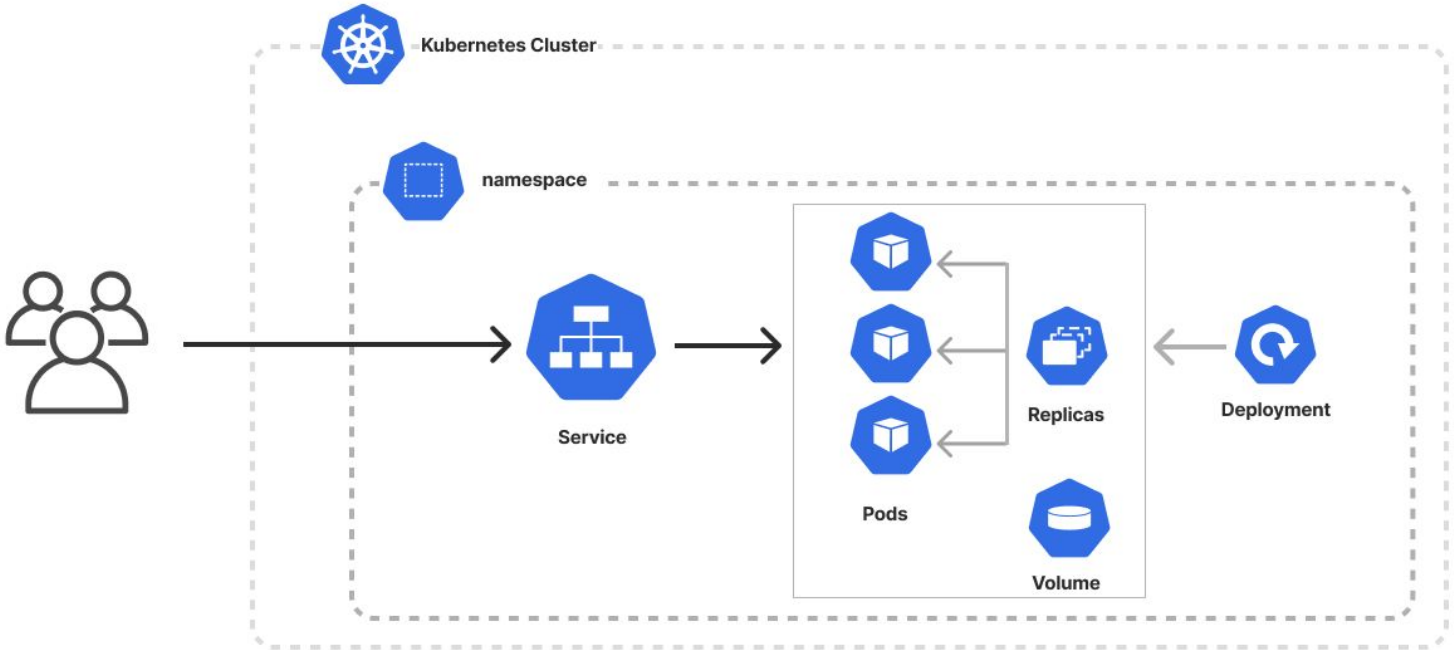


Estrutura hierárquica

- Resumo
  - Pod: executa
  - ReplicaSet: mantém quantidade
  - Deployment: gerencia versões
  - Service: expõe acesso



Estrutura hierárquica



Ingress

- Responsável por expor aplicações HTTP/HTTPS para fora do cluster, funcionando como uma camada de roteamento
- Cada aplicação precisa de um Service (NodePort ou LoadBalancer)
- Um único ponto de entrada
- Fluxo
  - Cliente -> Ingress -> Service -> Pod
- Exemplos:
  - Traefik, NGINX, …

Rollout e Rollback

- Rollout
  - Processo de atualizar uma aplicação

kubectl set image deployment/node-deployment node=node:20

  - Kubernetes NÃO troca tudo de uma vez
    - Cria novos pods
    - Remove antigos gradualmente
    - Rolling Update

Rollout e Rollback

- Rollout
  - kubectl rollout status deployment/node-deployment
  - kubectl rollout history deployment/node-deployment
- Rollback
  - kubectl rollout undo deployment/node-deployment

```
kubectl annotate deployment node-deployment \
  kubernetes.io/change-cause="Atualizando imagem para node:20"
deployment.apps/node-deployment annotated
kubectl rollout history deployment/node-deployment
deployment.apps/node-deployment
REVISION  CHANGE-CAUSE
1         <none>
2         Atualizando imagem para node:20
```

Comando importantes

- A maioria dos comandos segue o padrão

```
kubectl <verbo> <recurso> [nome]
```

- Recursos
  - pod, service (svc), deployment (deploy), replicaset (rs), namespace (ns), node
- Verbos
  - get, describe, logs, delete

Comando importantes

- kubectl apply -f deployment.yaml
- kubectl create deployment nginx --image=nginx
- kubectl edit deployment node-deployment
- kubectl patch deployment node-deployment -p '{"spec":{"replicas":3}}'
- kubectl scale deployment node-deployment --replicas=4
- kubectl set image deployment/node-deployment node=node:20

Comando importantes

- kubectl exec -it node-app -- bash
- kubectl exec node-app -- ls
- kubectl port-forward svc/node-service 3000:80
- kubectl rollout status deployment/node-deployment
- kubectl rollout history deployment/node-deployment
- kubectl rollout undo deployment/node-deployment
- kubectl run NAME -it --image ubuntu -- /bin/bash
- kubectl exec -it teste -- /bin/bash

Nossa primeira aplicação

- Criação deployment
  - Label
    - node-app
  - Porta interna da aplicação
    - 3000

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: node-app
  template:
    metadata:
      labels:
        app: node-app
    spec:
      containers:
        - name: node
          image: diegocbcastroo/node-pod-demo:1.0
          ports:
            - containerPort: 3000
```

Nossa primeira aplicação

- Criação do service
  - porta da aplicação
    - 3000
  - Porta do pod
    - 30001
  - Porta do service
    - 80
    - kubectl exec -it <algum-pod> -- sh
    - curl http://node-service:80

```yaml
apiVersion: v1
kind: Service
metadata:
  name: node-service
spec:
  type: LoadBalancer
  selector:
    app: node-app
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30001
```

Nossa primeira aplicação

- Criação do ingress
  - Regra de acesso
  - Redirect para o service na porta

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: node-ingress
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: node-service
                port:
                  number: 80
```

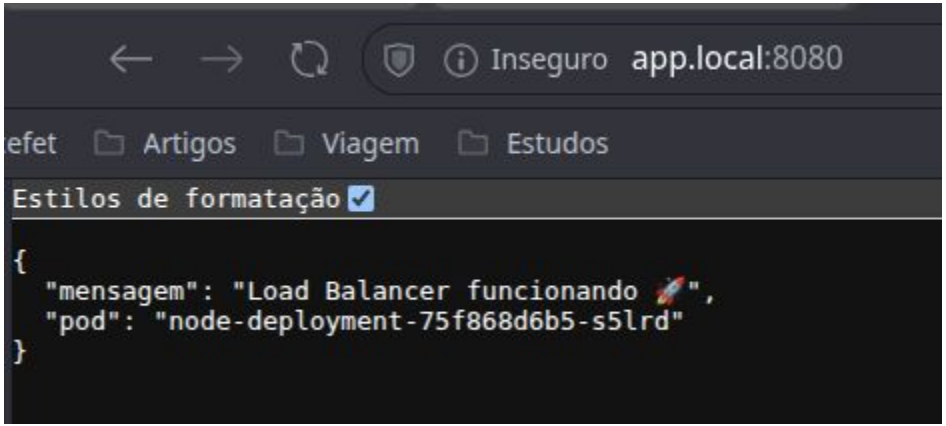
Nossa primeira aplicação

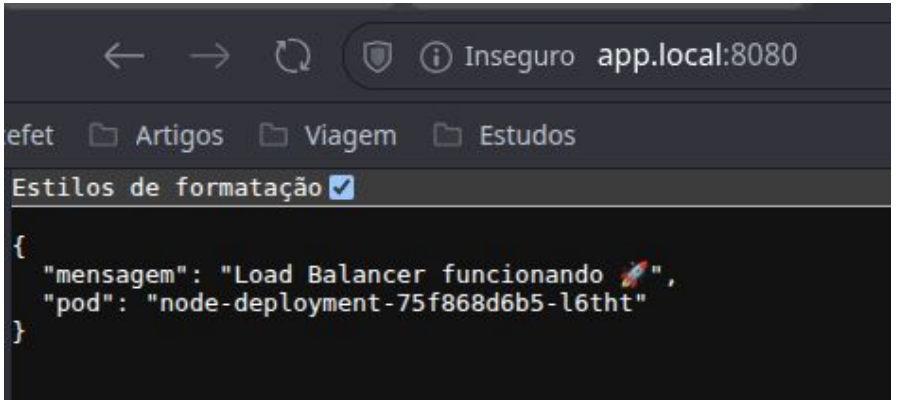
```
k3d cluster create meucluster --servers 1 --agents 2 \
-p "8080:80@loadbalancer"
```

```
cat <<EOF | sudo tee -a /etc/hosts
127.0.0.1 app.local
EOF
```

```
kubectl apply -f ./k8s
```

Nossa primeira aplicação





Balanceamento de carga entre os pods da aplicação